MODULE 1: DATABASE DESIGN AND IMPLEMENTATION

UNIT 2: ADVANCED SQL

2.1 Introduction:

Structured Query Language (SQL) is the most widely used commercial relational database language. It was designed for managing data in Relational Database Management System (RDBMS). It was originally developed at IBM in the SEQUEL XRM and System-R projects (1974-1977). Almost immediately, other vendors introduced DBMS products based on SQL. SQL continues to evolve in response to changing needs in the database area. This unit explains how to use SQL to access and manipulate data from database systems like MySQL, SQL Server, MS Access, Oracle, Sybase, DB2, and others

2.1. OBJECTIVES

At the end of this unit, students should be able to:

- understand the SQL statements
- be able to query a database using SQL queries

2.2 Basics Concepts of SQL

SQL - Structured Query Language is a standard language for accessing and manipulating databases. SQL lets you access and manipulate databases. It is used for defining tables and integrity constraints and for accessing and manipulating data. SQL. This unit explains how to use SQL to access and manipulate data from database systems like MySQL, SQL Server, MS Access, Oracle, Sybase, DB2, and others. Application programs may allow users to access a database without directly using SQL, but these applications themselves must use SQL to access the database.

Although SQL is an ANSI (American National Standards Institute) standard, there are many different versions of the SQL language. However, to be compliant with the ANSI standard, they all support at least the major commands (such as SELECT,

UPDATE, DELETE, INSERT, WHERE) in a similar manner. Most of the SQL database programs also have their own proprietary extensions in addition to the SQL standard.

The SQL language has several aspects to it.

The Data Manipulation Language (DML): This subset of SQL allows users to pose queries and to insert, delete, and modify rows. Queries are the main focus of this unit. We covered DML commands to insert, delete, and modify rows

The Data Definition Language (DDL): This subset of SQL supports the creation, deletion, and modification of definitions for tables and views. Integrity constraints can be defined on tables, either when the table is created or later. Although the standard does not discuss indexes, commercial implementations also provide commands for creating and deleting indexes.

Triggers and Advanced Integrity Constraints: The new SQL:1999 standard includes support for triggers, which are actions executed by the DBMS whenever changes to the database meet conditions specified in the trigger.

Embedded and Dynamic SQL: Embedded SQL features allow SQL code to be called from a host language such as C or COBOL. Dynamic SQL features allow a query to be constructed (and executed) at run-time.

Client-Server Execution and Remote Database Access: These commands control how a client application program can connect to an SQL database server, or access data from a database over a network.

Transaction Management: Various commands allow a user to explicitly control aspects of how a transaction is to be executed.

Security: SQL provides mechanisms to control users' access to data objects such as tables and views.

Advanced features: The SQL:1999 standard includes object-oriented features, recursive queries, decision support queries, and also addresses emerging areas such as data mining, spatial data, and text and XML data management.

2.3. HISTORY OF SQL

SQL was developed by IBM Research in the mid 70's and standardized by the ANSI and later by the ISO. Most database management systems implement a majority of one of these standards and add their proprietary extensions. SQL allows the retrieval, insertion, updating, and deletion of data. A database management system also includes management and administrative functions. Most – if not all –implementations also include a command-line interface (SQL/CLI) that allows for the entry and execution of the language commands, as opposed to only providing an application programming interface (API) intended for access from a graphical user interface (GUI).

The first version of SQL was developed at IBM by Andrew Richardson, Donald C. Messerly and Raymond F. Boyce in the early 1970s. This version, initially called **SEQUEL**, was designed to manipulate and retrieve data stored in IBM's original relational database product; System R. IBM patented their version of SQL in 1985, while the SQL language was not formally standardized until 1986 by the American National Standards Institute (ANSI) as SQL-86. Subsequent versions of the SQL standard have been released by ANSI and as International Organization for Standardization (ISO) standards.

Originally designed as a declarative query and data manipulation language, variations of SQL have been created by SQL database management system (DBMS) vendors that add procedural constructs, flow-of-control statements, user-defined data types, and various other language extensions. With the release of the SQL: 1999 standard, many such extensions were formally adopted as part of the SQL language via the SQL Persistent Stored Modules (SQL/PSM) portion of the standard. SQL was adopted as a standard by ANSI in 1986 and ISO in 1987.

In a nutshell, SQL can perform the following

1. SQL can execute queries against a database

2. SQL can retrieve data from a database
3. SQL can insert records in a database
4. SQL can update records in a database
5. SQL can delete records from a database
6. SQL can create new databases
7. SQL can create new tables in a database
8. SQL can create stored procedures in a database
9. SQL can create views in a database
10. SQL can set permissions on tables, procedures, and views

2.4 The Form of a Basic SQL Query

The basic form of an SQL query is as follows:

SELECT [DISTINCT] *select-list*
FROM *from-list*
WHERE *qualification*

Every query must have a SELECT clause, which specifies columns to be retained in the result, and a FROM clause, which specifies a cross-product of tables. The optional WHERE clause specifies selection conditions on the tables mentioned in the FROM clause.

2.4.1 The Syntax of a Basic SQL Query

1. The from-list in the FROM clause is a list of table names. A table name can be followed by a range variable; a range variable is particularly useful when the same table name appears more than once in the from-list.
2. The select-list is a list of (expressions involving) column names of tables named in the from-list. Column names can be prefixed by a range variable.
3. The qualification in the WHERE clause is a Boolean combination (i.e., an expression using the logical connectives AND, OR, and NOT) of conditions of the form expression op expression, where op is one of the comparison operators {<, <=,

=, <>, >=, >}. An expression is a column name, a constant, or an (arithmetic or string) expression.

4. The *DISTINCT* keyword is optional. It indicates that the table computed as an answer to this query should not contain duplicates, that is, two copies of the same row. The default is that duplicates are not eliminated.

2.5. SQL STATEMENTS

Most of the actions you need to perform on a database are done with SQL statements. Some database systems require a semicolon at the end of each SQL statement. Semicolon is the standard way to separate each SQL statement in database systems that allow more than one SQL statement to be executed in the same call to the server. We are using MS Access and SQL Server 2000 and we do not have to put a semicolon after each SQL statement, but some database programs force you to use it. SQL statements can be divided into two parts:

SQL statements are basically divided into four; viz;

1. Data Manipulation Language (DML)
2. Data Definition Language (DDL)
3. Data Control Language (DCL)
4. Transaction Control

2.5.1 DATA MANIPULATION LANGUAGE (DML)

DML retrieves data from the database, enters new rows, changes existing rows, and removes unwanted rows from tables in the database, respectively. The basic DML includes the following;

1. Select statement
2. Insert statement
3. Update statement
4. Delete statement
5. Merge statement

DATA DEFINITION LANGUAGE sets up, changes and removes data structures from tables. The basic Data Definition Language includes the following;

1. Create statement
2. Alter statement
3. Drop statement
4. Rename statement
5. Truncate statement
6. Comment statement

DATA CONTROL LANGUAGE (DCL) gives or removes access rights to both a database and the structures within it. The basic DCLs are;

1. Grant Statement
2. Revoke Statement

TRANSACTION CONTROL manages the changes made by the DML statements. Changes to the data can be grouped together into logical transactions. The basic Transaction control languages are;

1. Commit
2. Rollback
3. Save point

Using the following simple rules and guidelines, you can construct valid statements that are both easy to read and easy to edit

1. SQL statements are not case sensitive, unless indicated
2. SQL statements can be entered on one or many lines
3. Keywords cannot be split across lines or abbreviated
4. Clauses are usually placed on separate lines for readability
5. Indents should be used to make code readable
6. Keywords typically are entered in uppercase; all other words, such as table names and columns are entered in lowercase

2.6 Viewing the Structure of a Table

The structure of any database table can be view by using the describe clause of the SQL statement. The general syntax of the *describe* statement is given below;

DESCRIBE *table;*

For the purpose of this course two tables called Departments and Employees in the Oracle database will be used. Thus, we need to see the structure of this tables so that we will be able to familiarize ourselves with the column used in the table. To do this, we write the query;

DESCRIBE *departments;*

NAME	NULL?	TYPE
DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	CARCHAR(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

From the table above, we can infer that departments table has 4 columns and that 2 of these columns are not allowed to be null.

DESCRIBE *employees;*

Name	Null?	Type
EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

From the table above, we can infer that employees table has 11 columns and that 5 of these columns are not allowed to be null.

2.7 SQL SELECT STATEMENTS

To extract data from the database, you need to use the SQL SELECT statement. You may need to restrict the columns that are displayed. Using a SELECT statement, you can do the following;

1. **Projection:** You can use the projection capability to choose the columns in a table that you want to return by your query. You can choose as few or as many columns of the table as you require.
2. **Selection:** You can use the selection capability in SQL to choose the rows in a table that you want to return by a query. You can use various criteria to restrict the rows that you use.
3. **Joining:** You can use the join capability to bring together data that is stored in different tables by creating a link between them.

SUMMARY OF THE FUNCTIONS OF SQL

1. SQL can execute queries against a database
2. SQL can retrieve data from a database
3. SQL can insert records in a database
4. SQL can update records in a database
5. SQL can delete records from a database
6. SQL can create new databases
7. SQL can create new tables in a database
8. SQL can create stored procedures in a database
9. SQL can create views in a database
10. SQL can set permissions on tables, procedures, and views
11. SQL can allow the construction codes manipulating database

2.7.1 Using SQL for Web Site

To build a web site that shows some data from a database, you will need the following:

1. An RDBMS database program (i.e. MS Access, SQL Server, MySQL)

2. A server-side scripting language, like PHP or ASP
3. SQL

1. HTML / CSS

Relational Database Management System (RDBMS)

RDBMS is the basis for SQL, and for all modern database systems like MS SQL Server, IBM DB2, Oracle, MySQL, and Microsoft Access. The data in RDBMS is stored in database objects called tables. A table is a collection of related data entries and it consists of columns and rows.

2.8 SQL SYNTAX

Database Tables

A database most often contains one or more tables. Each table is identified by a name (e.g. "Customers" or "Orders"). Tables contain records (rows) with data.

Below is an example of a table called "Persons":

P_Id	LastName	FirstName	Address	City
1	Akinbode	Ola	10, Odeku Str.	Lagos
2	Okafor	Chris	23, Princewill Drive	Port Harcourt
3	Amodu	Ali	20, Dauda lane	Kaduna

The table above contains three records (one for each person) and five columns (P_Id, LastName, FirstName, Address, and City).

Format of SQL Statements

Most of the actions you need to perform on a database are done with SQL statements. The following SQL statement will select all the records in the "Persons" table:

```
SELECT * FROM Persons
```

Some database systems require a semicolon at the end of each SQL statement. Semicolon is the standard way to separate each SQL statement in database systems that allow more than one SQL statement to be executed in the same call to the server.

SQL, DML and DDL

SQL can be divided into two parts: The Data Manipulation Language (DML) and the Data Definition Language (DDL).

The query and update commands form the DML part of SQL:

SELECT - extracts data from a database **UPDATE** - updates data in a database

DELETE - deletes data from a database.

INSERT INTO - inserts new data into a database

The DDL part of SQL permits database tables to be created or deleted. It also define indexes (keys), specify links between tables, and impose constraints between tables. The most important DDL statements in SQL are:

CREATE DATABASE - creates a new database

ALTER DATABASE – modifies a database

CREATE TABLE - creates a new table

ALTER TABLE - modifies a table

DROP TABLE - deletes a table

CREATE INDEX - creates an index (search key)

DROP INDEX - deletes an index

The CREATE TABLE Statement

The CREATE TABLE statement is used to create a table in a database.

SQL CREATE TABLE Syntax

```
CREATE TABLE table_name
(
column_name1 data_type,
column_name2 data_type,
column_name3 data_type,
....
)
```

The data type specifies what type of data the column can hold. For a complete reference of all the data types available in MS Access, MySQL, and SQL Server visit www.datatyperef.com

CREATE TABLE Example

Now we want to create a table called "Persons" that contains five columns:

P_Id, LastName, FirstName, Address, and City. We use the following

CREATE TABLE statement:

```
CREATE TABLE Persons
(
P_Id int,
LastName varchar(255),
FirstName varchar(255),
    Address varchar(255),
City varchar(255)
)
```

The P_Id column is of type int and will hold a number. The LastName, FirstName, Address, and City columns are of type varchar with a maximum length of 255 characters.

The empty "Persons" table will now look like this:

P Id	LastName	FirstName	Address	City

The empty table can be filled with data with the INSERT INTO statement.

2.8 The SQL SELECT Statement

The SELECT statement is used to select data from a database.

The result is stored in a result table, called the result-set.

SQL SELECT Syntax

```
SELECT column_name(s)
FROM table_name
```

and

```
SELECT * FROM table_name
```

Note: *SQL is not case sensitive. SELECT is the same as select.*

An SQL SELECT Example

The "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Akinbode	Ola	10, Odeku Str.	Lagos
2	Okafor	Chris	23, Princewill Drive	Port Harcourt
3	Amodu	Ali	20, Dauda lane	Kaduna

Now we want to select the content of the columns named "LastName" and "FirstName" from the “Persons” table above. We use the following SELECT statement:

SELECT LastName, FirstName FROM Persons

The result-set will look like this:

LastName	**FirstName**
Akinbode	Ola
Okafor	Chris
Amodu	Ali

SELECT * Example

Now we want to select all the columns from the "Persons" table.

We use the following SELECT statement:

SELECT * FROM Persons

Tip: *The asterisk (*) is a quick way of selecting all columns!*

The result-set will look like this:

P_Id	LastName	FirstName	Address	City
1	Akinbode	Ola	10, Odeku Str.	Lagos
2	Okafor	Chris	23, Princewill Drive	Port Harcourt
3	Amodu	Ali	20, Dauda lane	kaduna

Navigation in a Result-set

Most database software systems allow navigation in the result-set with programming functions, like:

Move-To-First-Record, Get-Record-Content, Move-To-Next-Record, etc.

The SQL SELECT DISTINCT Statement

In a table, some of the columns may contain duplicate values. This is not a problem; however, sometimes you will want to list only the different (distinct) values in a table.

The DISTINCT keyword can be used to return only distinct (different) values.

SQL SELECT DISTINCT Syntax

```
SELECT DISTINCT column_name(s)
FROM table_name
```

SELECT DISTINCT Example

The "PersonsOne" table:

P_Id	LastName	FirstName	Address	City
1	Akinbode	Ola	10, Odeku Str.	Lagos
2	Okafor	Chris	23, Princewill Drive	Port Harcourt
3	Amodu	Ali	20, Dauda lane	kaduna

Now we want to select only the distinct values from the column named "City" from the table above.

We use the following SELECT statement:

```
SELECT DISTINCT City FROM Persons
```

The result-set will look like this:

City
Lagos
Kaduna

SQL *WHERE Clause*

The WHERE clause is used to filter records. The WHERE clause is used to extract only those records that fulfill a specified criterion.

SQL WHERE Syntax

SELECT column_name(s)

FROM table_name

WHERE column_name operator value

WHERE Clause Example

The "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Akinbode	Ola	10, Odeku Str.	Lagos
2	Okafor	Chris	23, Princewill Drive	Porthacourt
3	Amodu	Ali	20, Dauda lane	Kaduna

Now we want to select only the persons living in the city *"Sandnes"* from the table above.

We use the following SELECT statement:

```
SELECT * FROM Persons
WHERE City='Kaduna'
```

The result-set will look like this:

P_Id	LastName	FirstName	Address	City
1	Amodu	Ali	20, Dauda lane	Kaduna

Quotes Around Text Fields

SQL uses single quotes around text values (most database systems will also accept double quotes). Although, numeric values should not be enclosed in quotes.

For text values:

```
SELECT * FROM Persons WHERE
FirstName='Chris'
```

This is wrong:

```
SELECT * FROM Persons WHERE FirstName=Chris
```

For numeric values:

This is correct:

```
SELECT * FROM Persons WHERE Year=1965
```

This is wrong:

```
SELECT * FROM Persons WHERE Year='1965'
```

Operators Allowed in the WHERE Clause

With the WHERE clause, the following operators can be used:

Operator	Description
=	Equal
<>	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
BETWEEN	Between an inclusive range

LIKE	Search for a pattern
IN	If you know the exact value you want to return for at least one of the columns

Note: In some versions of SQL the <> operator may be written as !=

SQL AND & OR Operators

The AND & OR operators are used to filter records based on more than one condition. The AND operator displays a record if both the first condition and the second condition is true while the OR operator displays a record if either the first condition or the second condition is true.

AND Operator Example

The "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Akinbode	Ola	10, Odeku Str.	Lagos
2	Okafor	Chris	23, Princewill Drive	Porthacourt
3	Amodu	Ali	20, Dauda lane	Kaduna

OR Operator Example

Now we want to select only the persons with the first name equal to "Tove" OR the first name equal to "Ola":

We use the following SELECT statement:

```
SELECT * FROM Persons
WHERE FirstName='Chris'
OR FirstName='Ali'
```

The result-set will look like this:

P_Id	LastName	FirstName	Address	City
2	Okafor	Chris	23, Princewill Drive	Porthacourt
3	Amodu	Ali	20, Dauda lane	Kaduna

Combining AND & OR

You can also combine AND and OR (use parenthesis to form complex expressions). Now we want to select only the persons with the last name equal to "Svendson" AND the first name equal to "Tove" OR to "Ola":

We use the following SELECT statement:

```
SELECT * FROM Persons WHERE
LastName='Akinbode' OR
LastName=’Okafor’
```

The result-set will look like this:

P_Id	LastName	FirstName	Address	City
1	Akinbode	Ola	10, Odeku Str.	Lagos
2	Okafor	Chris	23, Princewill Drive	Porthacourt

THE ORDER BY Keyword

ORDER BY Syntax

The ORDER BY keyword is used to sort the result-set. The ORDER BY keyword is used to sort the result-set by a specified column. The ORDER BY keyword sorts the records in ascending order by default. If you want to sort the records in a descending

order, you can use the DESC keyword. The order by clause comes last in a select statement.

Syntax of the order by clause given below;

SELECT *expr*
FROM *table*
[WHERE *condition(s)*]
[ORDER BY{*column, expr}* [ASC|DESC]

ORDER BY EXAMPLE

Sorting in Descending order

SELECT last_name, job_id, department_id, hire_date

FROM employees

ORDER BY last

LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
Akinbode	SA_REP	80	21-APR-19
Amodu	SA_REP	20	21-APR-19
Buba	SA_REP	80	24-MAR-19
Ngozi	ST_CLERK	50	08-MAR-19
Okafor	SA_REP	80	23-FEB-19
Sowale	ST-CLERK	50	06-FEB-19

We also sort using multiple columns.

For example,

SELECT last_name, department_id, salary FROM

employees ORDER BY department_id, salary

DESC;

2.9 INSERT STATEMENT

INSERT INTO Syntax

The **INSERT INTO** statement is used to insert new records statement is used to insert a new row in a table.

The syntax of the insert statement is;

INSERT IBTO *table [{column, [,column…..])]*
VALUES (*value [, value….]);*

In the syntax,

table is the name of the table
column is the name of the column
v*alue* is the corresponding value for the column

INSERT STATEMENT EXAMPLE

INSERTING NEW ROWS

Example:

INSERT INTO departments (department_id, department_name,

manager_id, location_id) VALUES (170, 'Public Relations',100,1700);

INSERTING ROWS WITH NULL VALUES

Implicit method example

INSERT INTO departments (department_id, department_name)
Values (30,'Purchasing');
Explicit Method example
INSERT INTO departments
Values (100, 'Finance', NULL, NULL);

INSERTING SPECIAL VALUES

Example:

INSERT INTO employees (employee_id, first_name, last_name, email, phone_number,

hire_date, job_id,salary, commission_pct, manager_id, department_id)

Values (7, 'Adeola', 'Chalse', 'ade_char', '2348039990985', SYSDATE,

'AC_ACCOUNT', 6900, NULL, 205, 100);

2.10 UPDATE STATEMENT

UPDATE STATEMENT Syntax

The UPDATE statement is used to update existing records in a table.

```
UPDATE table_name
SET column1=value, column2=value2,...
WHERE some_column=some_value
```

Note: *Notice the WHERE clause in the UPDATE syntax. The WHERE clause specifies which record or records that should be updated. If you omit the WHERE clause, all records will be updated!*

SQL UPDATE EXAMPLES

updating rows in a table

```
UPDATE employees
SET department_ID=70
WHERE employee_ID = 113;
```

SQL DELETE Statement

SQL DELETE SYNTAX

The DELETE statement is used to delete records and rows in a table

```
DELETE FROM table_name
WHERE some_column=some_value
```

SQL DELETE EXAMPLES

```
DELETE *
FROM employees;

DELETE FROM employees
Where department_id =60;
```

DELETE ALL ROWS

It is possible to delete all rows in a table without deleting the table. This means that the table structure, attributes, and indexes will be intact:

```
DELETE FROM table_name

or

DELETE * FROM table_name
```

Note: *Once records are deleted, they are gone. Undoing this statement is not possible!*

2.11 Joining Tables

The Select statement can be used to join two tables together. It can be used to extract part of Table A and part of Table B to form Table C. For example, assuming *student* and *studentclass* are two different tables. Let us examine this instruction:-

```
Select student.SID, student.name,
studentclass.classname From student,
studentclass
Where student.SID = studentclass.SID
```

This statement shows that SID, name are columns or fields from student table and classname and SID are also columns from studentclass table.

The fields in the new table to form by this instruction are:-

SID name classname

2.12 ARITHMETIC OPERATIONS

Create expressions with number and date data by using arithmetic operators. You may need to modify the way in which data is displayed, perform calculations, or look at *what-if* scenarios. These are all possible using arithmetic expressions. An arithmetic expression can contain column names, constant numeric values and arithmetic operators.

Operator	Description
+	Add
-	Subtract
*	Multiply
/	Divide

USING ARITHMETIC OPERATORS

Let us first extend Table persons to Table employees by adding salary column to it and adding three more records. For subsequent illustrations in this unit, the table is also assumed to have more than 5 columns.

The example below describes a scenario in which arithmetic operators can be used.

SELECT last_name, salary, salary+300

FROM employees;

This gives

LastName	Salary	Salary+300
Akinbode	4800	5100
Okafor	17000	17300
Amodu	12000	12300
Buba	9000	9300
Ngozi	7700	8000
Sowale	24000	24300

2.13 OPERATOR PRECEDENCE

If an arithmetic expression contains more than one operator, multiplication and division are evaluated first. If operators within an expression are of the same priority, then evaluation is done from left to right. Parenthesis can be used to force the expression within parentheses to be evaluated first. Multiplication and division take priority over addition and subtraction. Example of a query that shows how operator precedence works is shown below;

SELECT *last_name, salary, 12*salary+100*

FROM employees

LastName	**Salary**	**12*Salary+100**
Akinbode	4800	57700
Okafor	17000	2014100
Amodu	12000	144100
Buba	9000	108100
Ngozi	7700	92500
Sowale	24000	288100

A query that uses brackets to override the operator precedence is shown below;

SELECT last_name, salary, 12*(salary+100)

FROM employees

LastName	**Salary**	**12*(Salary+100)**
Akinbode	4800	58800
Okafor	17000	205200
Amodu	12000	145200
Buba	9000	109200
Ngozi	7700	93600
Sowale	24000	289200

DEFINING A NULL VALUE

A null is a value that is unavailable, unassigned, unknown, or inapplicable. If a row lacks the data value for a particular column, that value is said to be null, or to contain a null. Columns of any data type can contain nulls. However, some constraints, NOT NULL and PRIMARY KEY, prevent nulls from being used in the column.

A query that shows the null values is shown below;

SELECT *last_name, job_id, salary, commission_pct*

FROM *employees;*

LAST_NAME	JOB_ID	SALARY	COMMISSION_PCT
Akinbode	ST_MAN	4800	
Okafor	ST_CLERK	17000	
Amodu	ST_CLERK	12000	
Buba	ST_CLERK	9000	
Ngozi	ST_CLERK	7700	
Sowale	ST_CLERK	24000	

In the COMMISSION_PCT column in the EMPLOYEES table, notice that only a sales manager (form the job_id column) can earn a commission.

Null Values in Arithmetic Expressions

Arithmetic expressions containing a null value evaluate to null.

Example:

SELECT last_name, 12*salary*commission_pct

FROM employees;

LAST_NAME	12*SALARY*COMMISSION_PCT
Akinbode	
Okafor	
Amodu	
Buba	
Ngozi	
sowale	

Processing Multiple Tables

1. Join–a relational operation that causes two or more tables with a common domain to be combined into a single table or view.

2. Equi-join–a join in which the joining condition is based on equality between values in the common columns; common columns appear redundantly in the result table.
3. Natural join–an equi-join in which one of the duplicate columns is eliminated in the result table.
4. Outer join–a join in which rows that do not have matching values in common columns are nonetheless included in the result table (as opposed to *inner* join, in which rows must have matching values in order to appear in the result table)
5. Union join–includes all columns from each table in the join, and an instance for each row of each table
6. Self join–Matching rows of a table with other rows from the same table

TIPS FOR DEVELOPING QUERIES

1. Be familiar with the data model (entities and relationships)
2. Understand the desired results
3. Know the attributes desired in result
4. Identify the entities that contain desired attributes
5. Review ERD
6. Construct a WHERE equality for each link
7. Fine tune with GROUP BY and HAVING clauses if needed
8. Consider the effect on unusual data

Query Efficiency Considerations

1. Instead of SELECT *, identify the specific attributes in the SELECT clause; this helps reduce network traffic of result set
2. Limit the number of subqueries; try to make everything done in a single query if possible
3. If data is to be used many times, make a separate query and store it as a view

Guidelines for Better Query Design

1. Understand how indexes are used in query processing
2. Keep optimizer statistics up-to-date
3. Use compatible data types for fields and literals
4. Write simple queries
5. Break complex queries into multiple simple parts
6. Don't nest one query inside another query
7. Don't combine a query with itself (if possible avoid self-joins)
8. Create temporary tables for groups of queries
9. Combine update operations
10. Retrieve only the data you need
11. Don't have the DBMS sort without an index
12. Learn!
13. Consider the total query processing time for ad hoc queries

DATA DICTIONARY FACILITIES

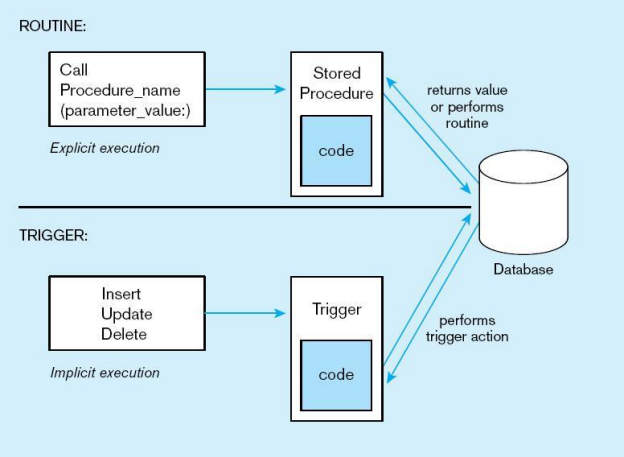
1. System tables that store metadata
2. Users usually can view some of these tables
3. Users are restricted from updating them
4. Some examples in Oracle 11g

Table	Description
DBA_TABLES	Describes all tables in the database
DBA_TAB_COMMENTS	Comments on all tables in the database
DBA_CLUSTERS	Describes all clusters in the database
DBA_TAB_COLUMNS	Describes columns of all tables, views, and clusters
DBA_COL_PRIVS	Includes all grants on columns in the database
DBA_COL_COMMENTS	Comments on all columns in tables and views
DBA_CONSTRAINTS	Constraint definitions on all tables in the database
DBA_USERS	Information about all users of the database

TRIGGERS AND ROUTINES

1. Triggers–routines that execute in response to a database event (INSERT, UPDATE, or DELETE)
2. Routines
    1. Program modules that execute on demand
3. Functions–routines that return values and take input parameters
4. Procedures–routines that do not return values and can take input or output parameters

Triggers contrasted with stored procedures



Trigger syntax in SQL

```
CREATE TRIGGER trigger_name
        {BEFORE | AFTER | INSTEAD OF} {INSERT | DELETE | UPDATE} ON
        table_name
        [FOR EACH {ROW | STATEMENT}] [WHEN (search condition)]
        <triggered SQL statement here>;
```

2.14 Some of the most important SQL Commands

1. **SELECT** - extracts data from a database
2. **UPDATE** - updates data in a database

3.	**DELETE**	-	deletes data from a database
4.	**INSERT INTO**	-	inserts new data into a database
5.	**CREATE DATABASE**	-	creates a new database
6.	**ALTER DATABASE**	-	modifies a database
7.	**CREATE TABLE**	-	creates a new table
8.	**ALTER TABLE**	-	modifies a table
9.	**DROP TABLE**	-	deletes a table
10.	**CREATE INDEX**	-	creates an index (search key)
11.	**DROP INDEX**	-	deletes an index

EMBEDDED AND DYNAMIC SQL

1. Embedded SQL

1. Including hard-coded SQL statements in a program written in another language such as C or Java

2. Dynamic SQL

1. Ability for an application program to generate SQL code on the fly, as the application is running

REASONS TO EMBED SQL IN 3GL

1. Can create a more flexible, accessible interface for the user
2. Possible performance improvement
3. Database security improvement; grant access only to the application instead of users

2.15 Conclusion

When we wish to extract information from a database, we communicate with the Database Management System (DBMS) using a query language called SQL. SQL is the most frequently used programming language in the world, in the sense that every day, more SQL programs are written, compiled and executed than programs in any other computer programming language. SQL is used with *relational* database systems. In a

relational database, all of the data is stored in tables. SQL has many built-in functions for performing calculations on data.

2.16 Tutor Marked Assignment

1. Describe the six clauses in the syntax of an SQL query, and show what type of constructs can be specified in each of the six clauses. Which of the six clauses are required and which are optional
2. What is a view in SQL, and how is it defined?

2.17 References and Further Reading:

David M. Kroenke, David J. Auer (2008). Database Concepts. New Jersey . Prentice Hall

Elmasri Navathe (2003). Fundamentals of Database Systems. England. Addison Wesley.

Fred R. McFadden, Jeffrey A. Hoffer (1994). Modern Database management. England. Addison Wesley Longman

Graeme C. Simsion, Graham C. Witt (2004). Data Modeling Essentials. San Francisco. Morgan Kaufmann

Pratt Adamski, Philip J. Pratt (2007). Concepts of Database Management. United States. Course Technology.